Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н. Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика, искусственный интеллект и системы управления» (ИУ)
КАФЕДРА «Информационная безопасность» (ИУ8)

Лабораторная работа № 10

ПО КУРСУ

«Алгоритмические языки»

на тему «Entity Framework, Microsoft SQL Server»

Студент ИУ8-31 (Группа) Р. Д. Хан (И. О. Фамилия)

Преподаватель Р. Д. Исянов (И.О. Фамилия)

2026 г.

Цель работы

Изучить ORM Entity Framework и хранение котировок в SQL-базе с определением изменения цены.

Условие задачи

1. Создать приложение для загрузки цен акций из ЛР9 и сохранения в базу данных. По введённому тикеру вывести, выросла или упала цена по сравнению с предыдущим днём.

2. Заполнять базу данными Market Data API. Использовать предложенную схему либо собственную; допускается любая SQL-база. После заполнения анализировать последнюю цену и предыдущую и записывать итог в TodaysCondition. Графический интерфейс необязателен.

Описание решения

Используются Entity Framework Core 8.0.15 и SQLite, разрешённая условием альтернатива SQL Server. Контекст StocksContext содержит Stocks, StockPrices и TodaysConditions. EnsureCreatedAsync создаёт базу. Внешние ключи связывают цены и состояния с тикером; уникальные индексы запрещают дубли тикера, пары тикер-дата и итогового состояния.

MarketClient загружает дневные свечи за год. Для каждого тикера новые даты добавляются, существующие обновляются. Затем выбираются две последние даты торгов. Сравниваются цены Close; результат «выросла», «упала» либо «не изменилась» сохраняется в TodaysConditions. Если есть только один день, данных для сравнения недостаточно.

В выходные или при задержке котировок используются последние доступные торговые дни. Их даты показаны явно: программа не называет устаревшую свечу сегодняшней. Просмотр сохранённых состояний возможен без повторного обращения к API. Учебный режим использует отдельную stocks_demo.db.

Режим --sample загружает публичный AAPL из API и использует отдельную stocks_sample.db. Режим --demo работает на вымышленных свечах; основной режим использует полный ticker.txt и stocks.db.

Схема базы данных

Stocks: Id, Ticker. StockPrices: Id, StockId, Date, High, Low, Close. TodaysConditions: Id, StockId, Date, PreviousDate, LastPrice, PreviousPrice, Condition. Связь Stocks → StockPrices: один ко многим; Stocks → TodaysConditions: один к одному.

Порядок запуска

Восстановить NuGet-пакеты и запустить Lab10. Ввести «да» для загрузки, затем тикер для просмотра. Для API задать MARKETDATA_TOKEN. Режим --demo

использует отдельную учебную базу; тикеры TEST_UP, TEST_DOWN и TEST_SAME демонстрируют три результата.

Код программы на языке C#

Файл MarketDataCore/Candle.cs

```
namespace MarketDataCore
{
    public class Candle
    {
        public DateTime Date { get; set; }
        public decimal High { get; set; }
        public decimal Low { get; set; }
        public decimal Close { get; set; }
    }
}
```

Файл MarketDataCore/MarketClient.cs

```
using System.Globalization;
using System.Net.Http.Headers;
using System.Text.Json;
namespace MarketDataCore
{
    public class MarketClient : IDisposable
    {
        private HttpClient client = new HttpClient();
        public MarketClient(string token)
        {
            client.Timeout = TimeSpan.FromSeconds(30);
            if (!string.IsNullOrWhiteSpace(token))
                client.DefaultRequestHeaders.Authorization =
                    new AuthenticationHeaderValue("Bearer", token);
        }
        public static List<Candle> Parse(string json)
        {
            using (JsonDocument doc = JsonDocument.Parse(json))
            {
                JsonElement root = doc.RootElement;
                string status = root.GetProperty("s").GetString();
                if (status != "ok")
                    throw new InvalidOperationException("Статус API: " + status);
                JsonElement times = root.GetProperty("t");
                JsonElement highs = root.GetProperty("h");
                JsonElement lows = root.GetProperty("l");
                JsonElement closes = root.GetProperty("c");
                int count = times.GetArrayLength();
                if (count == 0 || highs.GetArrayLength() != count
                    || lows.GetArrayLength() != count || closes.GetArrayLength() != count)
                    throw new InvalidOperationException("Пустые или несогласованные массивы
свечей.");
                List<Candle> result = new List<Candle>();
                for (int i = 0; i < count; i++)
                {
                    Candle candle = new Candle();
                    candle.Date =
DateTimeOffset.FromUnixTimeSeconds(times[i].GetInt64()).UtcDateTime.Date;
                    candle.High = highs[i].GetDecimal();
                    candle.Low = lows[i].GetDecimal();
                    candle.Close = closes[i].GetDecimal();
                    result.Add(candle);
                }
                return result;
            }
        }
        public async Task<List<Candle>> LoadAsync(string ticker, DateTime from, DateTime to)
        {
            string url = "https://api.marketdata.app/v1/stocks/candles/D/"
                + Uri.EscapeDataString(ticker) + "/?from="
                + from.ToString("yyyy-MM-dd", CultureInfo.InvariantCulture)
```

```
                + "&to=" + to.ToString("yyyy-MM-dd", CultureInfo.InvariantCulture);
            using (HttpResponseMessage response = await client.GetAsync(url))
            {
                if (!response.IsSuccessStatusCode)
                    throw new InvalidOperationException("HTTP " + (int)response.StatusCode);
                return Parse(await response.Content.ReadAsStringAsync());
            }
        }
        public static decimal Average(List<Candle> candles)
        {
            if (candles.Count == 0) throw new ArgumentException("Нет свечей.");
            decimal sum = 0;
            foreach (Candle c in candles) sum += (c.High + c.Low) / 2;
            return sum / candles.Count;
        }
        public void Dispose() { client.Dispose(); }
    }
}
```

Файл Lab10/Models.cs

```
namespace Lab10
{
    public class Stock
    {
        public int Id { get; set; }
        public string Ticker { get; set; }
    }
    public class StockPrice
    {
        public int Id { get; set; }
        public int StockId { get; set; }
        public DateTime Date { get; set; }
        public decimal High { get; set; }
        public decimal Low { get; set; }
        public decimal Close { get; set; }
    }
    public class TodaysCondition
    {
        public int Id { get; set; }
        public int StockId { get; set; }
        public DateTime Date { get; set; }
        public DateTime PreviousDate { get; set; }
        public decimal LastPrice { get; set; }
        public decimal PreviousPrice { get; set; }
        public string Condition { get; set; }
    }
}
```

Файл Lab10/StocksContext.cs

```
using Microsoft.EntityFrameworkCore;
namespace Lab10
{
    public class StocksContext : DbContext
    {
        private string path;
        public DbSet<Stock> Stocks { get; set; }
        public DbSet<StockPrice> StockPrices { get; set; }
        public DbSet<TodaysCondition> TodaysConditions { get; set; }
        public StocksContext(string path) { this.path = path; }
        protected override void OnConfiguring(DbContextOptionsBuilder options)
        {
            options.UseSqlite("Data Source=" + path);
        }
        protected override void OnModelCreating(ModelBuilder model)
        {
            model.Entity<Stock>().HasIndex(s => s.Ticker).IsUnique();
            model.Entity<StockPrice>().HasIndex(p => new { p.StockId, p.Date }).IsUnique();
            model.Entity<TodaysCondition>().HasIndex(c => c.StockId).IsUnique();
            model.Entity<StockPrice>().HasOne<Stock>().WithMany()
                .HasForeignKey(p => p.StockId);
            model.Entity<TodaysCondition>().HasOne<Stock>().WithOne()
```

```
                .HasForeignKey<TodaysCondition>(c => c.StockId);
        }
    }
}
```

Файл Lab10/StockService.cs

```
using MarketDataCore;
using Microsoft.EntityFrameworkCore;
namespace Lab10
{
    public class StockService
    {
        public static async Task SaveAsync(StocksContext db, string ticker, List<Candle> candles)
        {
            Stock stock = await db.Stocks.FirstOrDefaultAsync(s => s.Ticker == ticker);
            if (stock == null)
            {
                stock = new Stock { Ticker = ticker };
                db.Stocks.Add(stock);
                await db.SaveChangesAsync();
            }
            foreach (Candle candle in candles)
            {
                StockPrice price = await db.StockPrices.FirstOrDefaultAsync(
                    p => p.StockId == stock.Id && p.Date == candle.Date);
                if (price == null)
                {
                    price = new StockPrice { StockId = stock.Id, Date = candle.Date };
                    db.StockPrices.Add(price);
                }
                price.High = candle.High;
                price.Low = candle.Low;
                price.Close = candle.Close;
            }
            await db.SaveChangesAsync();
            List<StockPrice> last = await db.StockPrices.Where(p => p.StockId == stock.Id)
                .OrderByDescending(p => p.Date).Take(2).ToListAsync();
            TodaysCondition condition = await db.TodaysConditions
                .FirstOrDefaultAsync(c => c.StockId == stock.Id);
            if (last.Count < 2)
            {
                if (condition != null) db.TodaysConditions.Remove(condition);
                await db.SaveChangesAsync();
                return;
            }
            if (condition == null)
            {
                condition = new TodaysCondition { StockId = stock.Id };
                db.TodaysConditions.Add(condition);
            }
            condition.Date = last[0].Date;
            condition.PreviousDate = last[1].Date;
            condition.LastPrice = last[0].Close;
            condition.PreviousPrice = last[1].Close;
            if (condition.LastPrice > condition.PreviousPrice) condition.Condition = "выросла";
            else if (condition.LastPrice < condition.PreviousPrice) condition.Condition = "упала";
            else condition.Condition = "не изменилась";
            await db.SaveChangesAsync();
        }
    }
}
```

Файл Lab10/Program.cs

```
using MarketDataCore;
using Microsoft.EntityFrameworkCore;
namespace Lab10
{
    internal class Program
    {
        static async Task Main(string[] args)
```

```
        {
            Console.OutputEncoding = System.Text.Encoding.UTF8;
            bool demo = args.Contains("--demo");
            bool sample = !demo && args.Contains("--sample");
            string path = demo ? "stocks_demo.db" : sample ? "stocks_sample.db" : "stocks.db";
            string data = Path.Combine(AppContext.BaseDirectory, "Data");
            using (StocksContext db = new StocksContext(path))
            {
                await db.Database.EnsureCreatedAsync();
                Console.WriteLine(demo ? "Учебная БД, не реальные котировки." : "База: " + path);
                if (sample) Console.WriteLine("Публичный пример API: только AAPL.");
                Console.Write("Загрузить цены? (да/нет): ");
                if (Console.ReadLine() == "да")
                {
                    string token = Environment.GetEnvironmentVariable("MARKETDATA_TOKEN");
                    if (!demo && !sample && string.IsNullOrWhiteSpace(token))
                    {
                        Console.WriteLine("Для загрузки задай MARKETDATA_TOKEN.");
                    }
                    else
                    {
                        string[] tickers = sample ? new string[] { "AAPL" } :
File.ReadAllLines(Path.Combine(data,
                            demo ? "ticker_demo.txt" : "ticker.txt"))
                            .Select(t => t.Trim()).Where(t => t != "").Distinct().ToArray();
                        using (MarketClient client = new MarketClient(token))
                        {
                            foreach (string ticker in tickers)
                            {
                                try
                                {
                                    List<Candle> candles;
                                    if (demo)
                                        candles = MarketClient.Parse(File.ReadAllText(
                                            Path.Combine(data, "Demo", ticker + ".json")));
                                    else
                                        candles = await client.LoadAsync(ticker,
                                            DateTime.Today.AddYears(-1), DateTime.Today);
                                    await StockService.SaveAsync(db, ticker, candles);
                                    Console.WriteLine(ticker + ": сохранено свечей " +
candles.Count);
                                }
                                catch (Exception e)
                                {
                                    Console.WriteLine(ticker + ": " + e.Message);
                                    if (e.Message == "HTTP 401" || e.Message == "HTTP 403"
                                        || e.Message == "HTTP 429") break;
                                }
                                if (!demo) await Task.Delay(1100);
                            }
                        }
                    }
                }
                Console.WriteLine("Строк: акции=" + await db.Stocks.CountAsync()
                    + ", цены=" + await db.StockPrices.CountAsync()
                    + ", состояния=" + await db.TodaysConditions.CountAsync());
                while (true)
                {
                    Console.Write("Тикер (пустая строка — выход): ");
                    string ticker = Console.ReadLine()?.Trim().ToUpperInvariant();
                    if (string.IsNullOrEmpty(ticker)) break;
                    Stock stock = await db.Stocks.FirstOrDefaultAsync(s => s.Ticker == ticker);
                    TodaysCondition condition = null;
                    if (stock != null)
                        condition = await db.TodaysConditions.FirstOrDefaultAsync(c => c.StockId ==
stock.Id);
                    if (condition == null)
                    {
                        Console.WriteLine("Нет данных для сравнения двух торговых дней.");
                        continue;
                    }
```

```
                    Console.WriteLine(ticker + ": " + condition.Condition);
                    Console.WriteLine(condition.PreviousDate.ToString("yyyy-MM-dd") + ": "
                        + condition.PreviousPrice + " -> " + condition.Date.ToString("yyyy-MM-dd")
                        + ": " + condition.LastPrice);
                    if (condition.Date.Date != DateTime.Today)
                        Console.WriteLine("Это последние доступные торговые данные, не котировка за
сегодня.");
                }
            }
        }
    }
}
```

Файл Lab10/Lab10.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <OutputType>Exe</OutputType>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>disable</Nullable>
  </PropertyGroup>
  <ItemGroup><ProjectReference Include="../MarketDataCore/MarketDataCore.csproj" /></ItemGroup>
  <ItemGroup><None Include="../Data/**" Link="Data/%(RecursiveDir)%(Filename)%(Extension)"
CopyToOutputDirectory="PreserveNewest" /></ItemGroup><ItemGroup>
    <PackageReference Include="Microsoft.EntityFrameworkCore.Sqlite" Version="8.0.15" />
  </ItemGroup>
</Project>
```

Результаты выполнения программы

Публичный пример AAPL загружен из API без токена и сохранён в SQLite именно через EF Core: 1 акция, 250 цен и 1 состояние. Close за 2026-10-29 — 348,91, за 2026-10-30 — 344,63; результат «упала». Поскольку 2026-11-01 — выходной, программа явно показывает последние торговые даты. Отдельно на учебной базе проверены рост, падение, отсутствие изменения, неизвестный тикер и повторный импорт без дублей: число строк осталось 3/6/3. Полный сбор по исходному списку требует пользовательского токена.

Lab10_sample_run.txt

```
База: stocks_sample.db
Публичный пример API: только AAPL.
Загрузить цены? (да/нет): AAPL: сохранено свечей 250
Строк: акции=1, цены=250, состояния=1
Тикер (пустая строка — выход): AAPL: упала
2026-10-29: 348.91 -> 2026-10-30: 344.63
Это последние доступные торговые данные, не котировка за сегодня.
Тикер (пустая строка — выход):
```

Lab10_run.txt

```
Учебная БД, не реальные котировки.
Загрузить цены? (да/нет): TEST_UP: сохранено свечей 2
TEST_DOWN: сохранено свечей 2
TEST_SAME: сохранено свечей 2
Строк: акции=3, цены=6, состояния=3
Тикер (пустая строка — выход): TEST_UP: выросла
2026-01-01: 10 -> 2026-01-02: 12
Это последние доступные торговые данные, не котировка за сегодня.
Тикер (пустая строка — выход): TEST_DOWN: упала
2026-01-01: 20 -> 2026-01-02: 18
Это последние доступные торговые данные, не котировка за сегодня.
Тикер (пустая строка — выход): TEST_SAME: не изменилась
2026-01-01: 30 -> 2026-01-02: 30
Это последние доступные торговые данные, не котировка за сегодня.
Тикер (пустая строка — выход): Нет данных для сравнения двух торговых дней.
Тикер (пустая строка — выход):
```

Lab10_repeat.txt

```
Учебная БД, не реальные котировки.
Загрузить цены? (да/нет): TEST_UP: сохранено свечей 2
TEST_DOWN: сохранено свечей 2
TEST_SAME: сохранено свечей 2
Строк: акции=3, цены=6, состояния=3
Тикер (пустая строка — выход): TEST_UP: выросла
2026-01-01: 10 -> 2026-01-02: 12
Это последние доступные торговые данные, не котировка за сегодня.
Тикер (пустая строка — выход):
```

Выводы по работе

Реализованы загрузка котировок из API, сохранение через EF Core и запрос изменения цены. Проверена настоящая история AAPL из 250 свечей. Дополнительная учебная проверка подтвердила все три результата сравнения и отсутствие дублей при повторном импорте.

Документация

Market Data: https://www.marketdata.app/docs/api/stocks/candles/ и https://www.marketdata.app/docs/api/authentication/

Microsoft Learn: https://learn.microsoft.com/ef/core/ ; OpenWeather: https://openweathermap.org/current